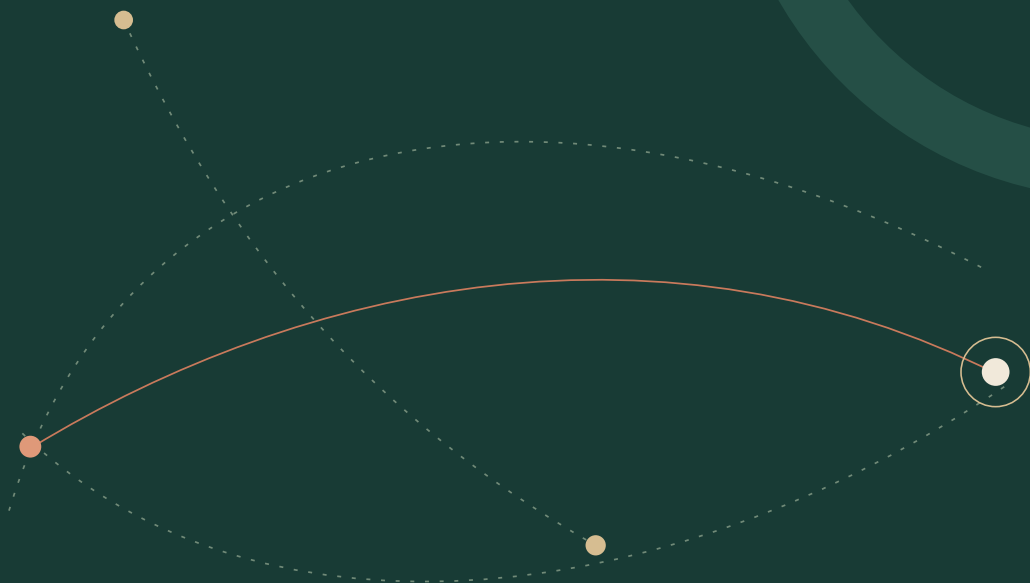




Systems that keep *their promises.*

A FIELD GUIDE TO BUILDING
DEPENDABLE DATA PRODUCTS

FIELDNOTES · FIRST DIGITAL EDITION · 2026

Systems That Keep Their Promises

A field guide to building dependable data products

CESAR PEDRIN

Copyright © 2026 Cesar Pedrin. All rights reserved.

This is an original field guide about the design and operation of dependable data products.

First digital edition, 2026.

Contents

Before you begin 4

Promise before platform 6

Give every fact a home 10

Commit before you celebrate 14

Build the read you actually need 18

Let work travel as messages 22

Split without losing the plot 26

Choose where truth lives 29

Practice the failure 33

From promise to running system 37

A NOTE FROM CESAR

Before you begin

Why this guide starts with promises instead of technology.

OPENING NOTE · CESAR PEDRIN

A note from Cesar

Systems work is often introduced as a tour of technologies: a database, a queue, a cache, a cluster. Those tools matter, but they do not explain why one design is trustworthy and another is not. The difference usually starts with a promise: what a person expects the product to do, what the system can prove it has done, and what happens when a dependency goes quiet at the wrong moment.

This guide is about that distance between intention and evidence. It follows a small set of practical questions: What does the customer believe? Which record is authoritative? When is a change durable? How stale may a read be? Who owns a delayed task? Where does a rule get enforced? Can the team recover when a machine, process, or assumption fails?

The examples use Mesa Market, a fictional neighborhood food co-op. It is intentionally ordinary. Orders, pickup windows, receipts, and inventory estimates are enough to reveal the hard parts without requiring a giant company or a specialized industry. The choices are not recipes to copy; they are examples of how to connect a product promise to a technical boundary.

How to use this guide

Read the chapters in order if you are starting a system or revisiting its foundations. If you have a specific problem, jump to the chapter that matches it: retries and durable writes, read freshness, asynchronous workers, partition choices, coordination, or recovery. The final design walks through the ideas as one coherent service.

For each system you work on, try answering the design check at the end of every chapter. Write the answer down. If two people on the team give different answers, you have found a useful design conversation—not a failure of vocabulary.

This is an original, practical introduction, not a catalog of products and not a substitute for testing against your own workload. Start with the smallest arrangement that can keep the promises your users need. Measure where it falls short. Add complexity only when it buys a specific improvement you can observe.

— Cesar Pedrin

SYSTEMS THAT KEEP THEIR PROMISES · CESAR PEDRIN

01 · START WITH THE USER

Promise before platform

Turn a vague request for reliability into visible, testable promises.

CHAPTER 01 · 08 · CESAR PEDRIN

Start with the sentence a customer can understand

A team says, “We need a scalable order system.” That sentence contains a wish, not a design. Scalable for which workload? What does an order mean when a phone loses its connection after payment? Is a one-second-old stock count acceptable, or can it cause a customer to arrive for groceries that do not exist?

Begin with the promise the product needs to keep. A useful promise is observable from outside the system and specific enough to test. For a neighborhood pickup service, it might read: **Once we confirm an order, we will keep its pickup slot and either provide every confirmed item or contact the customer before pickup.** That statement is not a database choice. It is a boundary for every database choice that follows.

Promises often contain several dimensions that teams accidentally bundle together:

- **Correctness:** which outcomes must never happen? A confirmed order must not silently disappear.
- **Freshness:** how old may a displayed fact be? A browse page may lag by a few seconds; a final stock reservation may not.
- **Availability:** which operations should remain possible during a dependency failure?
- **Latency:** how long may a person reasonably wait for each operation?

- **Durability:** what does “saved” mean, and what failures must it survive?

These are separate dials. “Highly available and strongly consistent” is not yet a useful requirement; it leaves out which operation, during which failure, with what response. Put the promise next to the operation that makes it real.

Make the invisible measurable

Turn each promise into a small contract. A contract names the actor, action, outcome, and acceptable time or error boundary. For example:

Operation	User-visible outcome	Target
Browse a pickup window	Show an estimate and its update time	p95 under 350 ms; at most 10 seconds old
Submit an order	Return a durable confirmation or a clear retry-safe failure	p95 under 800 ms
Cancel before cutoff	Release the reservation exactly once from the customer's point of view	Complete within 2 seconds
View an order after submit	Show the confirmed items and pickup slot	Available immediately from the write path

The exact numbers are product decisions, not universal benchmarks. A team should be able to explain why a number exists and what a customer experiences when it is missed. A percentile is more revealing than an average: averages can hide a long tail of people waiting while most requests finish quickly. Track error rates and freshness alongside latency; a fast answer with stale inventory can be worse than a slower, honest answer.

A service-level indicator is the measurement. A service-level objective is the target. An error budget is the amount of tolerated miss over a window. These terms are useful only when connected to a decision: if the freshness objective is

repeatedly missed, should the team reduce promotional traffic, simplify the browse view, or invest in a faster projection pipeline?

Draw the boundary of a promise

A system cannot promise what it does not control. If pickup depends on a partner's inventory feed, separate the promise for the catalog estimate from the promise for a confirmed reservation. Display uncertainty honestly. “Usually available” and “reserved for you” must not be the same state with different colors.

Write down the failure behavior, not only the happy path. What does the customer see when the payment provider times out? What if the service commits the order but its response never reaches the browser? A timeout means the caller did not learn the outcome; it does not prove the operation did not happen. Designing a safe retry is part of keeping the promise.

For every critical operation, record:

- ¹ The invariant that must hold.
- ² The authority allowed to decide it.
- ³ The durable point after which success may be reported.
- ⁴ The user-visible behavior when that point cannot be reached.
- ⁵ The signal operators use to notice a miss.

This tiny checklist is often more valuable than an early diagram of ten services. It exposes incompatible expectations while they are still cheap to change.

Let evidence choose the machinery

Only now ask what workload the system must support. Estimate reads and writes per second, typical and worst-case record sizes, retention, bursts, and geographic reach. Separate current traffic from plausible growth. A system designed for a fantasy workload is not safer; it merely adds more moving parts before anyone can measure their value.

Start with the simplest arrangement that can honor the contract. A single transactional store may handle years of traffic. A cache, queue, search index, or partitioned cluster earns its place when a measured pressure or a clear failure boundary calls for it. Each additional component creates a new place where data can be delayed, duplicated, unavailable, or misunderstood.

The point is not to prefer small systems forever. It is to make complexity answer a question. When the question changes, revisit the design. The platform follows the promise; the promise does not have to imitate the platform.

Design check: Can a support engineer explain, in plain language, when an order is merely requested and when it is genuinely confirmed?

02 · SHAPE THE INFORMATION

Give every fact a home

Model ownership, identity, and change before choosing storage.

CHAPTER 02 · 08 · CESAR PEDRIN

Model the meaning before the container

Storage discussions tend to begin with nouns: table, document, key, cache. Begin instead with the facts the business recognizes. In a pickup service, an order has a customer, a chosen window, a set of requested items, and a lifecycle. A pickup window has a capacity. An item has a catalog description and may have separate quantities at multiple locations.

A model is a set of claims about the world. Good models make important rules easy to state and hard to violate. If a reservation is for one item at one location, represent those dimensions explicitly. If “available quantity” is derived from counted stock minus active reservations, decide whether it is stored as a number, computed on demand, or maintained as a projection. Every representation is a bet about which update must be easy and which mistake must be prevented.

A practical first pass is to list:

- **Entities:** things with identity and a lifecycle, such as an order or pickup window.
- **Events:** facts that happened, such as an order being confirmed or a reservation being released.
- **Values:** descriptive details that are meaningful only in context, such as an address snapshot on an order.
- **Invariants:** conditions that must be true across a change, such as a window never exceeding its confirmed capacity.

The same word can describe more than one concept. “Inventory” might mean a count from a supplier, units physically checked in, units promised to orders, or an estimate shown to browsers. Give each meaning a name. Ambiguous nouns eventually become ambiguous columns and contradictory dashboards.

Assign authority, not just ownership

For every important fact, name the component that is allowed to change it. The catalog may own product descriptions; the reservation service may own claims on available stock; the order record may preserve what the customer actually bought. Ownership is about decision rights, not merely which team maintains a table.

This matters when a fact is copied. A browse index can hold product names for fast searching, and an analytics warehouse can hold historic order lines. Those are useful copies, but they should not quietly become authorities. If a copied value disagrees with the owner, the design needs a known path to refresh it and a rule for what the user sees in the meantime.

Name the answer to three questions:

- ¹ Where is the authoritative version of this fact?
- ² Who may change it, and under which rule?
- ³ Which copies exist, how are they refreshed, and how stale may they be?

This is the beginning of a data contract. It prevents a common organizational failure: two services both believe they are entitled to make the final decision.

Give identity a lifetime

Identifiers should survive the movement of data between screens, processes, and storage systems. A stable order ID is not the same as a customer's email address, a database row number, or the order's current position in a list. Use opaque identifiers when the value crosses trust boundaries; avoid exposing sequential IDs if they reveal business volume or invite enumeration.

Identity also needs scope. A product code may be unique within one co-op but not across every co-op in a network. A pickup window might be identified by location and start time, but that pair becomes awkward if schedules change. Choose an ID that represents the entity's lifetime, then store mutable attributes separately.

For records that can be edited concurrently, keep a revision or update token. A request can say “change this address if the order is still at revision 12.” If another change already produced revision 13, the system can reject the stale edit rather than erase it. Versioning is a simple, visible tool for expressing the rule “do not overwrite a change you did not see.”

Treat schemas as agreements over time

Stored data outlives the code that first wrote it. Renaming a field is not a local refactor when old application versions, delayed messages, exports, and backups still contain the earlier shape. Prefer changes that remain understandable during a transition: add an optional field, teach readers to accept both versions, backfill in bounded batches, then make the new field required only after old writers have gone away.

A safe migration often has four stages:

- ¹ Expand:** add a new representation without removing the old one.
- ² Dual-read or dual-write selectively:** bridge old and new clients while monitoring disagreement.

- 3 **Migrate:** copy historical data in restartable batches, with a checkpoint and rate limit.
- 4 **Contract:** remove the old path only after the rollback window closes.

Not every change needs all four steps, but every change should answer what happens if deployment stops halfway. A migration that only works when all servers update simultaneously is fragile in a rolling deployment.

A schema describes bytes; a data contract describes meaning. Record both. Include units, time zone, null semantics, valid state transitions, and who is permitted to write. “Amount” without currency and “date” without time zone are not complete facts.

Preserve what the customer meant

Some data is a live reference; some is a historical snapshot. An order line should usually preserve the product name, unit price, and quantity as understood at confirmation time. If the catalog title changes tomorrow, an old receipt should not rewrite itself. Conversely, a shipping address may need both the original submitted value and the normalized value used by a delivery route.

Choose deliberately between a pointer to current truth and a snapshot of past truth. Many records need both. The distinction makes audits, support conversations, and later corrections far less mysterious.

Design check: For each field in a critical record, can you name its meaning, authority, unit, and behavior after the underlying business object changes?

03 · MAKE WRITES TRUSTWORTHY

Commit before you celebrate

Follow a write from request to durable record, including retries and uncertainty.

CHAPTER 03 · 08 · CESAR PEDRIN

A request can have an uncertain ending

A browser sends “place this order.” The server validates it, commits the database transaction, and begins sending a response. The connection breaks before the browser receives that response. The order exists, but the customer sees a timeout. If the customer taps again, is the next request a duplicate order or a safe way to discover the first result?

This is one of the most important distinctions in distributed work: **a missing response is not evidence of a missing effect**. The caller knows only that it did not learn the outcome. A trustworthy write path must make uncertainty survivable.

A typical write has a sequence of meaningful stages:

- ¹ Accept and validate the request.
- ² Check authorization and business preconditions.
- ³ Apply the state change atomically with related facts.
- ⁴ Make the change durable under the promised failure model.
- ⁵ Return a result that names what was committed.
- ⁶ Publish follow-up work without creating a gap between the record and the message.

The exact implementation depends on the store, but the boundary must be clear. If the response says “confirmed,” identify the durable point that justifies those words.

Make retries recognizable

A client-generated idempotency key gives repeated attempts a stable identity. The server stores the key with the result of the first accepted attempt. When the same key arrives again, it returns that result rather than performing the action twice. A key should be scoped to the caller and operation, have a retention period long enough for realistic retries, and be associated with a request fingerprint so that accidental reuse with different content is rejected.

Idempotency does not mean every operation is naturally safe to repeat. “Set quantity to 4” is different from “add 4.” “Reserve this window for order 8F...” is different from “reserve one more slot.” Prefer commands that name the desired state or a unique operation. For event-like actions, assign an event or command ID and record whether it has already been applied.

Suppose the co-op receives the same order request twice. A unique constraint on the idempotency key can make the database the final arbiter, even when two application servers race. The losing request reads the stored outcome. This avoids relying on a process-local cache, which disappears on restart and cannot coordinate independent servers.

Define the durability boundary

Durability is not one universal checkbox. It depends on what failures the system promises to withstand. A local commit may survive a process restart but not the loss of the machine. A replicated commit can survive a machine failure but may still be lost if acknowledgements are returned before replicas persist the record. A remote backup can protect against a regional incident but may lag behind live writes.

Ask specific questions:

- After success is returned, what hardware or service failures may occur without losing the record?
- Does the write need to be copied to another machine, another zone, or another region before acknowledgment?
- How much recent work may be lost in the worst acceptable recovery scenario?
- How long may recovery take, and who has rehearsed it?

A stronger durability target may add latency or reduce the ability to accept writes during a network split. That trade is a product decision with operational consequences, not a contest to choose the most impressive setting.

Keep related facts together

An order confirmation may need to create an order record, reserve a pickup slot, and record a message for downstream notifications. If these actions happen in separate systems, a crash can land between them. The order may exist without its notification; worse, the notification may go out before the order is durable.

Where one store can atomically update the essential facts, use a transaction for that boundary. For work that must reach another system, a transactional outbox is a useful bridge: write the business change and an outbox row in the same transaction, then have a worker publish the row. The worker may publish twice if it crashes after sending but before marking the row complete, so consumers must also recognize duplicate message IDs.

The outbox does not create one giant transaction across the network. It turns an invisible gap into explicit, retryable state. Monitor the age of the oldest unpublished item; a growing outbox is a symptom that the promise is drifting.

Report outcomes honestly

Avoid a response that says “success” while critical work is merely queued unless the user understands what remains pending. Use explicit states such as **requested**, **confirmed**, **cancelled**, and **needs_attention**. State names should describe business reality, not infrastructure details such as “message sent.”

When an operation is still in progress, return a durable operation ID that the client can poll or subscribe to. Make a retry return the same operation. If the final outcome is rejection, preserve enough context to explain why and whether another attempt could succeed.

A well-designed write path does more than persist bytes. It gives the system and its users a stable answer after interruptions, duplicates, and delayed responses.

Design check: If the client times out immediately after the database commits, can it safely retry and learn the original result without creating a second effect?

04 • SERVE USEFUL VIEWS

Build the read you actually need

Design queries, projections, caches, and freshness as one experience.

CHAPTER 04 • 08 • CESAR PEDRIN

A read path is part of the product

A product may store orders correctly and still feel unreliable if the customer cannot see the confirmed order, the picker cannot find the list, or support cannot explain what happened. Reads shape trust. Design them from the questions each person asks, not from whichever storage format happens to be convenient.

List the screens and decisions first. A shopper browsing a pickup window needs a compact availability estimate. A customer viewing a receipt needs the exact items and terms that were confirmed. A picker needs an ordered work list grouped by location. An analyst needs history across many weeks. These are different queries with different freshness and latency requirements.

One data model rarely serves all of them equally well. That is acceptable. Keep one clear authority for changes, and create read-optimized representations where measurements justify them. A projection can flatten related information, precompute totals, or group work for a specific task. It is a view of facts, not a second, competing authority.

Choose between live reads and projections

A live query reads current authoritative records at the moment of the request. It is useful when the result must reflect the latest committed state and the query remains efficient. As relationships and request volume grow, a live query may require many lookups or expensive aggregation.

A projection is updated after changes occur. It can make reads fast and simple, but it introduces a delay and another lifecycle to operate. The design should name that delay as a freshness contract rather than describing the view as “real time.” If a dashboard usually catches up in two seconds but can take thirty during a backlog, tell the user what the displayed timestamp means and alert on lag.

A projection consumer should be restartable. It needs a checkpoint, a way to replay changes, and a strategy for rebuilding from authoritative data. If a projection cannot be reconstructed, it is not merely a convenience; it has become a hidden source of truth.

Make stale data safe

Stale reads are not always dangerous. A product description can be minutes old without harming a transaction. A stock estimate can also be stale while browsing, provided the final reservation rechecks current availability. The key is to put the authoritative decision at the point where an irreversible promise is made.

Useful patterns include:

- Display a freshness timestamp for data that changes often.
- Label estimates differently from confirmed values.
- Revalidate critical preconditions at submit time.
- Return a version with the view so a later command can detect that it is outdated.
- Design the UI to recover gracefully when a final check rejects an optimistic estimate.

This allows the system to use fast, eventually refreshed views without pretending they are exact. The user sees the boundary between a helpful preview and a committed decision.

Cache with a reason

A cache reduces repeated work by keeping a copy close to the reader. It can improve latency and absorb bursts, but it adds another place to reason about freshness, memory limits, eviction, and failure. Cache only after measuring a bottleneck or identifying a cost worth avoiding.

For mostly immutable content, use versioned keys or long cache lifetimes. For mutable values, decide what event invalidates the cache, how missed invalidations are repaired, and what happens when the cache is empty. If a cache miss overloads the database, request coalescing or a brief fallback may be necessary. If the cache is unavailable, the system should either continue at reduced speed or fail clearly; it should not accidentally turn a recoverable cache problem into a total outage.

Be cautious with a cached answer that controls a scarce resource. It may be fine to show “about six boxes remain,” but a cached value should not authorize the sixth and seventh reservations simultaneously. The authoritative store must enforce that invariant.

Make pagination stable

Large lists should be read in bounded pages. Offset-based pagination is easy to understand but can become expensive and unstable when records are inserted or removed between requests. Cursor pagination uses a stable ordering and returns a marker for the next page. A useful cursor includes all ordering fields, for example creation time plus a unique order ID, so ties do not cause records to disappear or repeat.

For a snapshot-like experience, bind pages to a consistent revision or timestamp when the storage system supports it. For a live activity feed, explain that new entries may appear above the current cursor. Pagination is a product behavior as much as a query trick.

Instrument the answer, not just the database

Measure end-to-end read latency, result size, cache hit rate, projection lag, and error rate. Track the freshness of the data presented to the user, not just the time taken to retrieve it. A fast query over a projection that stopped consuming yesterday is a successful database request and a failed product experience.

Design check: Which displayed values may be estimates, and where does the system recheck the fact before promising something costly or irreversible?

05 · COORDINATE WITHOUT COUPLING

Let work travel as messages

Use asynchronous work without losing ownership, order, or recovery paths.

CHAPTER 05 · 08 · CESAR PEDRIN

Separate the request from its aftermath

Some work should finish before the system acknowledges the user. Other work can happen later: sending a receipt, refreshing a search view, preparing a picker list, or updating an analytics feed. Moving follow-up work out of the request path can make the product more responsive and can isolate failures, but it also creates a new contract: the work is accepted now and completed later.

A message usually represents either a command—“send this receipt”—or a fact—“order 8F was confirmed.” Commands ask one owner to act. Facts announce something that has already happened. Mixing the two makes it unclear who may change the truth. A useful event name describes a completed business fact in the past tense; a command names an intended action.

The producer should publish only facts it can defend. The consumer should own how those facts affect its own state. A notification worker can decide how to format an email, but it should not decide whether an order was truly confirmed.

Expect delay and duplication

A queue or log may deliver a message more than once. A worker can complete an action, crash before recording its checkpoint, then receive the same message after restart. A delivery system that advertises “at least once” is saying duplicates are possible while messages are retried until acknowledged; it is not saying every consumer action is magically safe.

Make the consumer idempotent. Store the processed message ID or use a natural uniqueness rule. If a notification should be sent once per order and notification type, record that key before or with the send workflow. For external providers that do not support idempotency, model the uncertainty and reconcile from provider receipts rather than assuming a timed-out request did nothing.

Ordering also needs a scope. A global order over every event in the company is expensive and often unnecessary. It may be enough to preserve order for changes to one order or one pickup window. Partition messages by the entity whose sequence matters, and include a version so the consumer can detect a gap or an out-of-order update.

Use backpressure instead of wishful speed

If messages arrive faster than workers can process them, backlog grows. Adding more workers helps only when the work can be parallelized and the downstream dependency can absorb the extra load. Otherwise it moves the queue into a database or external service and makes the overload harder to control.

Watch queue depth, oldest-message age, processing rate, retries, and failures by message type. Age often better captures customer harm than depth: a million cheap analytics events may be harmless, while a few delayed pickup cancellations may be urgent. Apply rate limits, bounded concurrency, and prioritization where promises differ.

A worker should make progress in small, restartable units. Long jobs need checkpoints. Poison messages need a bounded retry policy and a quarantine or dead-letter path with enough context for diagnosis. A dead-letter queue is not a graveyard; assign an owner, alert on new entries, and define how repaired messages are replayed safely.

Connect records to messages

Publishing an event directly after a database commit creates a gap. If the process crashes between the commit and publish, the record changes but subscribers never hear about it. If publish happens first, subscribers may act on a fact that the database later rejects.

An outbox, introduced earlier, records the event in the same transaction as the business change. A publisher moves outbox rows to the broker and can safely retry. Consumers still need idempotency, because the publisher may not know whether a previous send succeeded. The pattern does not eliminate failure; it makes recovery explicit at both ends.

For workflows across multiple owners, use a state machine with durable steps rather than pretending all participants share one transaction. A reservation workflow might request stock, wait for a confirmation, and either confirm the order or release the reservation if payment fails. Each transition should have a timeout, a retry rule, and a compensating action when appropriate.

Keep contracts evolvable

A message can be sitting in a queue for hours or replayed months later. Include a schema version or design fields so new consumers can ignore additions. Make changes compatible during rollout: deploy readers that tolerate the new shape before producers send it. Avoid reusing a field with a different meaning; old records do not learn the new interpretation.

Keep payloads small and intentional. A message may include a stable entity ID, event ID, version, occurrence time, and the minimum facts the consumer needs. If a consumer fetches current state by ID, it may see a later version than the event describes. Choose between a self-contained event snapshot and a pointer to current state based on whether the consumer needs history or only the latest answer.

Design check: If a worker receives the same message twice, what durable rule prevents the second delivery from producing a second business effect?

SYSTEMS THAT KEEP THEIR PROMISES · CESAR PEDRIN

06 · GROW WITH INTENTION

Split without losing the plot

Partition data and work around real traffic while planning for skew and change.

CHAPTER 06 · 08 · CESAR PEDRIN

Scale the hot path, not the diagram

A system can grow in several different ways: larger records, more active users, heavier queries, bursts at particular times, or higher availability expectations. Scaling is not a single operation. Identify which resource is constrained before splitting a dataset or adding another service.

Partitioning divides a collection or stream into pieces that can be stored or processed independently. A partition key decides where each piece goes. A good key spreads ordinary work while keeping operations that need to coordinate close together. The correct answer depends on the invariant and traffic pattern, not just on the number of rows.

For a co-op ordering service, order history might be partitioned by a stable order ID, while reservations could be grouped by location and pickup window because capacity decisions happen there. An analytics export might use date ranges. There is no rule that every dataset needs the same key.

Understand the shape of traffic

Average load hides hotspots. A popular item, one large customer, a flash sale, or a single pickup window can direct a large fraction of requests to one partition. This is skew. Adding partitions does not fix a key that funnels every request to the same place.

Measure both key frequency and resource use. A partition receiving many inexpensive reads may be fine; one receiving many contended updates may be saturated. Consider whether a hot read can be cached or replicated, whether a write can be queued, or whether the logical entity can be safely divided into sub-entities. Splitting a scarce-resource counter across shards is not safe if the product still promises an exact global limit without a coordination rule.

Hashing keys spreads unrelated records but makes range queries harder. Range partitioning helps scans over time or geography but can send new writes to the same newest partition. Composite keys can put frequently related work together while adding enough variation to reduce hotspots. Each choice makes some operations easier and others more expensive.

Keep cross-partition work visible

A query that touches one partition can be fast and predictable. A query that fans out to many partitions needs coordination, merge logic, timeouts, and partial-result behavior. If a customer asks for one order by ID, route directly. If an analyst asks for all orders across every region, use a path designed for broad scans rather than forcing the transactional store to impersonate an analytical warehouse.

Transactions across partitions may require additional coordination and can reduce availability during network failures. Before asking for them, check whether the invariant can be placed inside one partition, expressed as a reservation workflow, or settled asynchronously with a clear pending state. Some global invariants really do require coordination; the goal is to make that cost explicit rather than accidentally paying it on every request.

Plan for movement

Partition layouts change as traffic and storage grow. A migration may move records while reads and writes continue. A safe plan defines a routing epoch, tracks which version owns each key, and handles writes that arrive during the move. Depending on the system, this might mean copy-then-catch-up, temporary dual reads, or a service-managed rebalancing process.

Never assume that changing the number of hash buckets is a free configuration edit. It can remap a large share of keys. Keep data placement behind a routing layer or use a database whose rebalancing behavior is understood and observable. Test the movement with realistic key distributions and enough spare capacity to hold both old and new copies during transition.

Separate workload classes

Transactional updates, interactive browse queries, background projections, and analytical scans compete for different resources. They may share infrastructure at first, but give each a clear budget. A batch report should not consume every connection needed by checkout. A queue worker should slow down when its downstream store is near saturation. A cache should not hide an unbounded fan-out query.

Capacity planning is a feedback loop: measure, forecast, test, and revise. Keep a representative load test that includes skew, cold caches, and downstream latency. Record what fails first. A system is easier to scale when the team knows whether its next limit is CPU, storage, connections, lock contention, network bandwidth, or human operational capacity.

Design check: Which real operation benefits from your proposed partition key, and which one becomes more expensive because of it?

07 · COORDINATE CAREFULLY

Choose where truth lives

Place invariants, concurrency control, and conflict handling where they belong.

CHAPTER 07 · 08 · CESAR PEDRIN

Put each invariant beside its authority

An invariant is a rule that must hold across a change: an order cannot be both cancelled and fulfilled; a pickup window cannot exceed its confirmed capacity; a refund cannot exceed the amount collected. If multiple processes can update the facts that enforce the rule, those processes need a coordination mechanism.

The simplest strong boundary is often one transaction in one authoritative store. A conditional update such as “reserve one place only if remaining capacity is greater than zero” can serialize competing requests at the point where the decision is made. A version check can reject stale edits. A unique constraint can prevent duplicate external IDs even if application code races.

Move the invariant outward only when there is a concrete need. Splitting ownership across services may improve team autonomy or isolation, but it turns a local atomic change into a workflow. The workflow must represent intermediate states, retry decisions, and compensation. That is a meaningful architectural cost, not merely a network call replacing a function call.

Choose a concurrency strategy

Pessimistic coordination prevents simultaneous work from proceeding through a contested section, often by locking a record. It is easy to reason about when contention is modest and transactions are short. Long-held locks reduce throughput and can cause deadlocks, so keep the critical section narrow and define a timeout.

Optimistic coordination lets work proceed and checks at commit that the data has not changed. A revision number or compare-and-set condition can detect a conflict. It works well when conflicts are uncommon; frequent conflicts turn repeated retries into waste. Clients need a clear conflict response and a safe way to reload or merge.

Single-owner serialization routes all decisions for one key through one authority. It makes per-key ordering straightforward, but a hot key can become a bottleneck and the owner needs a failover story. It does not automatically provide global order across every key.

There is no universally best approach. The cost of a false acceptance, a rejected request, and a delayed request differs by domain. Choosing a strategy means choosing which outcome is acceptable under contention.

Treat network uncertainty as normal

A distributed service can be healthy while a particular request times out. A timeout limits how long a caller waits; it does not cancel work already accepted elsewhere. Retries can multiply load precisely when a dependency is struggling. Use bounded retries with backoff and jitter, respect idempotency keys, and place deadlines across the call chain so work does not continue long after the user has left.

A circuit breaker or load shedder can prevent repeated calls to a dependency that is already failing. It should fail in a way the product can explain. If browsing can continue but confirmation cannot, expose that distinction instead of declaring the whole service either “up” or “down.” A graceful degraded mode must not accidentally make a stronger promise than the normal mode.

Know what “consistent” means here

Different operations may need different visibility guarantees. A user who submits an order and immediately opens its receipt usually expects read-your-writes: their own committed change is visible. A public browse page may tolerate a short delay. A support report might need a stable snapshot so multiple pages tell one coherent story.

Name the consistency behavior for each path. Does a read come from the leader, a replica, or a projection? Can a recently committed update be temporarily absent? Does a response include a revision that the client can use to request at least that version? Precise language turns an abstract debate into a product contract.

When replicas are distributed across a network, a partition can separate nodes that can no longer communicate. If both sides accept conflicting updates, the system needs a conflict rule later. If only one side may make the decision, the other side must reject or delay some writes. This trade-off is not erased by a clever slogan. Decide which behavior is safe for the business operation.

Use conflict resolution only when the domain supports it

Some concurrent changes merge naturally. Two devices adding separate items to a shopping list can preserve both additions. A text field edited differently on two devices may need a human choice. Inventory reservations do not safely merge by summing two local “available” counts if doing so can oversell stock.

Automatic merge rules are business rules. State them in domain language, test them under duplicated and reordered updates, and preserve enough history to explain the result. If no principled merge exists, choose one authority or surface the conflict instead of silently picking a winner.

Design check: Name one invariant in your system. Which component is the final authority when two valid-looking requests try to change it at the same time?

SYSTEMS THAT KEEP THEIR PROMISES · CESAR PEDRIN

08 · OPERATE AND RECOVER

Practice the failure

Make health observable, rehearse recovery, and evolve the system without drama.

CHAPTER 08 · 08 · CESAR PEDRIN

Design for the ordinary bad day

Operations are not a phase that begins after architecture. A data system is defined partly by how it behaves when a database is slow, a worker is stuck, a disk fills, a certificate expires, or a deployment has to be rolled back. These are not exotic exceptions; they are normal conditions that a dependable product learns to survive.

For each critical dependency, write a short failure note:

- What symptom will a customer notice?
- What should continue to work, and what should stop?
- Which metric or alert reveals the problem?
- What is the first safe mitigation?
- What state might need repair after recovery?

A dependency map is useful only if people can use it under pressure. Include owners, dashboards, runbooks, and the boundary of each service's responsibility. When an alert fires, it should point to a user impact or a concrete risk, not merely report that a graph crossed an unexplained line.

Observe behavior at the right level

Logs help answer what happened to one request. Metrics reveal trends and saturation. Traces show where a request spent time across components. Events and audit records explain important business transitions. These signals complement one another; no single one replaces the rest.

Give each request a correlation ID that follows it across services, while avoiding secrets or personal data in log fields. Record state transitions with stable entity IDs and actor context. Use structured fields rather than relying on text that changes during refactoring. Sample high-volume traces deliberately, but preserve enough detail for rare failures and slow requests.

Monitor not only resource health but promises: confirmation latency, cancellation completion, projection freshness, oldest queue age, recovery point, and failed reservation rate. Alert on sustained customer harm and on early warning indicators that leave time to act. Every alert should have an owner and a next step.

Backups are plans, not files

A backup is useful only if it can be restored. Define a recovery point objective—the maximum amount of recent data the business can afford to lose—and a recovery time objective—the longest acceptable time to restore service. Measure both in exercises. A backup that has never been restored is an assumption.

Practice restoring into an isolated environment. Verify record counts, checksums, key relationships, and application compatibility. Test the sequence for restoring encryption keys and credentials as well as data. A backup can be intact and unusable if its format, keys, or schema have changed without a tested path back.

For projections and caches, decide whether to rebuild from the authoritative source or restore a snapshot and replay changes. For event histories, validate

retention and replay throughput. A recovery plan should explain which views will be temporarily stale and when they become trustworthy again.

Deploy changes with an exit path

A safe deployment is not one that can never fail; it is one that detects failure early and has a bounded rollback or forward-fix path. Use small changes, gradual rollout, health checks tied to the promised behavior, and a clear decision threshold. Feature flags can separate shipping code from enabling behavior, but each flag needs an owner and removal date.

Data migrations need special care because reverting code does not reverse durable writes. Expand-and-contract changes, restartable backfills, and compatibility windows let old and new versions coexist. Before a risky change, record the current state, expected signals, rollback trigger, and who can approve it. Afterward, verify the result rather than inferring success from a green deployment job.

Learn from incidents without blame

An incident review should reconstruct the conditions that made the failure possible, not search for one person to blame. Ask which assumptions were invisible, what signals were missing, why existing defenses did not contain the impact, and how the recovery could become easier. Then select a small number of changes with owners and dates.

The goal is not to prevent every error. It is to reduce the chance that one error becomes a broad outage, shorten the time to understand what is happening, and make recovery safe. Reliability grows through repeated practice: tabletop exercises, fault injection in controlled environments, restore drills, and post-deployment checks.

A mature system makes important failures boring. Operators can see the scope, customers receive an honest status, retries do not create duplicate effects, and

restored data can be verified. The best emergency procedure is one the team has already rehearsed when there was no emergency.

Design check: When was the last time the team restored a backup and proved the application could use it?

FIELD NOTES · A WORKED DESIGN

From promise to running system

Bring the ideas together in a small neighborhood co-op ordering service.

WORKED DESIGN · CESAR PEDRIN

The scenario: Mesa Market

Mesa Market is a fictional neighborhood food co-op with several pickup locations. Members browse weekly offerings, place an order, choose a pickup window, and collect a packed bag. The team is small. It needs a design it can operate before it needs a design that survives internet-scale traffic.

The first promise is deliberately narrow: **A confirmed order has a stable item list and pickup window. Mesa Market will not confirm more reservations than a location can fulfill. If a final check fails, the customer sees a clear alternative before the order is confirmed.** Browse quantities are estimates; the confirmation step is authoritative.

The promise identifies two different read and write paths. Browsing can use a refreshed projection. Confirmation must check the current reservation state. That one distinction keeps a fast browse page from pretending to reserve stock.

Choose the first authoritative boundary

Start with a relational database for the core records. This is not a claim that one database is always right; it is a reasonable first fit for related records and transactional invariants. The initial model includes:

- `pickup_window`: location, start time, capacity, and active state.

- **reservation**: window, order ID, quantity, and lifecycle state.
- **order**: member, selected window, confirmation state, and idempotency key.
- **order_line**: product ID plus a snapshot of name, unit, price, and quantity.
- **outbox**: event ID, order ID, event type, version, and publication state.

A request to confirm an order opens a short transaction. It verifies the window is open, conditionally increments its reserved quantity only if capacity remains, inserts the order and its lines, and records an **OrderConfirmed** outbox event. A unique constraint protects the idempotency key. If any precondition fails, the transaction commits no partial order. The service returns a confirmation only after the database reports a durable commit under the chosen configuration.

This design places the scarce-window invariant beside its authority. It also preserves what the member agreed to, even if catalog details change later. The implementation should test two simultaneous requests competing for the last place; a unit test of the happy path is not enough.

Add read models where they pay off

The member's order receipt can read the authoritative order record. The browse page needs a product and availability summary, so a small projection worker consumes catalog and reservation events and writes a denormalized browse view. Each view row includes an update time. The page labels availability as an estimate, and confirmation rechecks the reservation transaction.

An operations view can be another projection grouped by pickup window, showing confirmed item counts and packing status. It does not get to alter the order's confirmed lines. If it falls behind, the freshness metric alerts and the page shows “updated at” rather than quietly presenting an old count as current.

For the first release, both projections can live in the same database if load is modest. The data contract and rebuild process matter more than a premature choice of separate technology. A rebuild command reads authoritative records in

batches, replaces the projection safely, and tracks progress. The team can later move a projection to a dedicated search or analytical system without moving the authority for confirmations.

Make side effects durable and repeatable

The outbox publisher sends `OrderConfirmed` to a message broker. A receipt worker and the operations projection consume it independently. Every event has a stable ID and order version. If the publisher sends twice, consumers recognize the duplicate. If a consumer fails, the message remains retryable; after bounded retries, it is quarantined with an alert and a replay procedure.

The confirmation response does not wait for an email. The durable order is the confirmation; the email is a follow-up. The order page displays its current state directly from the authoritative record, so a delayed receipt cannot make the order disappear. A status page can distinguish “order confirmed” from “receipt email sent.”

If a payment provider participates, give that interaction its own state. A timeout from the provider is not the same as a decline. Use the provider's idempotency mechanism and reconcile uncertain attempts. Do not reserve a pickup window indefinitely while the outcome is unknown; define a reservation expiry and a visible pending state. The exact sequence depends on whether the co-op collects payment at checkout or at pickup, which is a business decision rather than a database preference.

Add capacity only when the evidence asks

Suppose traffic grows. First measure what is saturated: database connections, a hot pickup window, projection lag, or a slow product query. Add an index or connection pool adjustment only when evidence supports it. Cache stable catalog details if repeated reads dominate. Increase worker concurrency only if the broker backlog is growing and the database has headroom.

If several locations grow independently, location may be a useful operational partition for reservation work. But a member's order history then spans locations, so it may need a separate read model. A single pickup window remains a hot key during a popular release; exact capacity still requires one authority for that window. If contention becomes substantial, queue reservation commands by window or move that decision to a dedicated owner. Test the ordering and failover behavior before changing the promise.

None of these changes requires the browse projection to become authoritative or the entire application to be split into services. Keep the invariant and its owner stable while changing the paths around it.

Rehearse the uncomfortable cases

Before launch, test these scenarios:

- ¹ The client times out after a successful commit, then repeats the request with the same key.
- ² Two members request the last pickup place at the same moment.
- ³ The database commits the order, but the outbox publisher is offline for twenty minutes.
- ⁴ The projection worker sees the same event twice and later receives a newer version first.
- ⁵ A backup is restored and the browse view is rebuilt from authoritative records.
- ⁶ The payment provider accepts a charge but its response is lost.

Each test needs a defined customer result, a durable system state, and an operator signal. If the team cannot describe one of those, the design is not finished.

The compact blueprint

Mesa Market's first dependable version is not dependable because it uses a long list of technologies. It is dependable because its promises map to explicit boundaries:

- A transaction owns order confirmation and pickup capacity.
- An idempotency key makes uncertain retries safe.
- An outbox records follow-up work with the business change.
- Consumers can handle duplicates and expose their lag.
- Projections speed up reads but never authorize reservations.
- A recovery drill proves that core data and useful views can return.

That blueprint can begin as one application and one database. It can later gain queues, partitions, replicas, or service boundaries as real constraints appear. The shape may change; the promise remains the compass.

Final design check: Can every “confirmed” message in the product be traced to a durable decision, and can the team prove that by exercising the failure paths?